Пояснение к первичным артефактам

Курсовая работа «Архитектура программных систем»
Имитационная модель СМО · Вариант 3

Бизнес-домен: система обработки промышленных тревог

ИБ · И31 · П32 · Д1031 · Д1003 · Д2П1 · Д2Б3 · ОР1 · ОД1

Как пользоваться документом. Для каждой диаграммы сначала дана сама схема, затем пояснение: что на ней изображено и что происходит по шагам. Номера в квадратных скобках [N] совпадают с номерами стрелок на сиквенс-диаграммах. В конце собраны вопросы, которые обычно задают на защите, с короткими ответами.

Раздел	Что внутри
1. О чём работа	СМО, домен, расшифровка варианта, метод особых событий
2. Как читать диаграммы	Легенда UML: стрелки, блоки alt/loop/opt, обозначения в диаграмме классов
3. Общий сиквенс	Весь алгоритм модели с альтернативами, шаги [1]–[59]
4. Переполнение → обработка	Обязательный сценарий на конкретных числах, шаги [1]–[58]
5. Happy Flow	Буфер пуст, прибор свободен, шаги [1]–[50]
6. Краевые кейсы	Переход указателя по кольцу, выбивание, выбор прибора, шаги [1]–[54]
7. Диаграмма классов	Из каких классов состоит программа и как они связаны
8. Flowchart с дорожками	Тот же алгоритм в виде блок-схемы по участникам
9. ArchiMate	Бизнес-процесс обработки тревоги
10. Вопросы на защите	Короткие ответы на типичные вопросы преподавателя

1. О чём работа

Работа посвящена **имитационному моделированию системы массового обслуживания (СМО)**. Программа не ждёт реального времени, а «проигрывает» работу системы в модельном времени и считает, насколько хорошо система справляется с потоком задач: сколько задач теряется, сколько они ждут, насколько загружены исполнители. По этим цифрам потом подбирается оптимальная конфигурация: число исполнителей, размер очереди, скорость обработки.

Элементы любой СМО:

- **Источник** порождает заявки через случайные промежутки времени;
- **Буфер** — ограниченная очередь, где заявки ждут обработки;
- **Прибор** обслуживает одну заявку за раз;
- **Диспетчер постановки (ДП)** решает, куда положить новую заявку и кого выкинуть при переполнении;
- **Диспетчер выбора (ДВ)** решает, какую заявку взять из буфера и на какой прибор отправить.

Наш бизнес-домен

Промышленный объект. Датчики оборудования, контроллеры технологических линий и система контроля доступа (СКУД) присылают **тревожные сообщения**. Сообщения встают в общую ограниченную очередь, откуда их забирают **рабочие процессы анализа**: проверяют, классифицируют и регистрируют инцидент. Если очередь переполнена, удаляется **самая старая** тревога, потому что свежие сообщения важнее устаревших.

Элемент СМО	Сущность домена	Класс в коде
Источник	Датчик / контроллер линии / СКУД	Source
Заявка	Тревожное сообщение	Request
Буфер	Ограниченная очередь тревог	Buffer
Прибор	Рабочий процесс анализа тревог	Device
ДП	Компонент приёма тревог	PlacementDispatcher
ДВ	Планировщик проверки тревог	SelectionDispatcher
Отказ	Удаление тревоги без проверки	Request.reject() + статистика

Расшифровка варианта

Код	Что означает	Формула / правило
ИБ	Источники бесконечные: после каждой заявки планируют следующую	—
И31	Пуассоновский поток: интервалы между тревогами экспоненциальные	$\tau = -\ln(r) / \lambda$, $r \in (0; 1)$
П32	Время обработки распределено равномерно на [a; b]	$T_{обсл} = a + (b - a) \cdot r$
Д1О31	Постановка в буфер «по кольцу»	Поиск свободного места с указателя УП; затем УП := место, следующее за занятым
Д1ОО3	Отказ получает самая старая заявка в буфере	Вытесняется заявка с минимальным временем постановки; новая встаёт на её место

Код	Что означает	Формула / правило
Д2П1	Выбор прибора по приоритету номера	Перебор П1 → П2 → … → Пm, берётся первый свободный
Д2Б3	Выбор заявки из буфера «по кольцу»	Поиск занятого места с указателя УВ; затем УВ := место, следующее за выбранным
ОД1	Пошаговый режим	Календарь событий, буфер, указатели, приборы, % отказа на каждом шаге
ОР1	Автоматический режим	Сводная таблица результатов (графики не нужны)

Метод особых событий — главная идея модели

Модельное время не тикает равномерно, а **прыгает от события к событию**. Все будущие события лежат в **календаре** (очередь, отсортированная по времени). Программа в цикле берёт ближайшее событие, переводит часы на его время, обрабатывает его и, если нужно, добавляет в календарь новые события.

В календаре бывает всего два типа событий:

- **GENERATION** — источник прислал тревогу. Реакция: ДП кладёт её в буфер, затем ДВ проверяет, нет ли свободного прибора.
- **SERVICE_END** — прибор закончил проверку. Реакция: статистика учитывает обслуженную заявку, ДВ берёт из буфера следующую.

Все остальные события из пошагового режима (поставлена в буфер, выбита, выбрана, отправлена на прибор) происходят *внутри* обработки этих двух.

Триггеры диспетчеров. ДП запускается событием «сгенерирована тревога». ДВ запускается двумя событиями: «заявка поставлена в буфер» (иначе простаивающий прибор не узнает о новой заявке) и «прибор освободился». Сам по себе ни один диспетчер работу не начинает.

Инвариант модели. Если в буфере есть заявки, значит все приборы заняты: ДВ разбирает буфер сразу, как только появляется свободный прибор. Отсюда «буфер полон ⇒ все приборы заняты». Поэтому ДП может решать вопрос об отказе, ничего не зная о приборах.

2. Как читать диаграммы

Сиквенс-диаграммы

Обозначение	Смысл
Вертикальные колонки (линии жизни)	Участники: календарь, симулятор, источник, ДП, буфер, ДВ, приборы, статистика
Время	Идёт сверху вниз. То, что ниже, происходит позже
Сплошная стрелка →	Вызов метода / сообщение
Пунктирная стрелка ⇢	Ответ (Response): что вернулось вызывающему
Стрелка в себя	Внутреннее действие участника (например, сдвиг указателя)
[N]	Порядковый номер шага, по нему удобно ссылаться
Блок alt / else	Развилка: выполняется ровно одна из ветвей
Блок loop	Цикл: повторяется, пока выполняется условие
Блок opt	Необязательный фрагмент: выполняется только при условии
Жёлтая заметка	Расшифровка кода варианта или пояснение к шагу
== Заголовок ==	Разделитель смысловых фаз (например, новое событие календаря)

Диаграмма классов

Обозначение	Смысл
Прямоугольник	Класс (в Go — структура). Сверху поля, снизу методы
− перед именем	Приватный член (в Go — имя с маленькой буквы)
+ перед именем	Публичный член (в Go — имя с большой буквы)
Стрелка с подписью «- buffer»	Класс хранит ссылку на другой класс в приватном поле с этим именем
Закрашенный ромб ◆	Композиция: симулятор создаёт объект и владеет им
Пустой ромб ◇	Агрегация: объект хранит другие объекты (буфер хранит заявки)
Пунктир «create»	Класс создаёт объекты другого класса
Пунктир с треугольником	Реализация интерфейса
1, 0..1, 1..m, 0..N, *	Кратность: сколько объектов на другом конце связи
«...» над именем	Стереотип: роль класса в СМО или в домене

Обозначения в примерах

- **И2-5** — пятая тревога от источника И2;
- **УП** — указатель постановки (у ДП), **УВ** — указатель выбора (у ДВ);
- **[1] И2-5 | [2] – | [3] И3-4** — содержимое мест буфера, «–» означает, что место пустое;
- **Тбп** — время в буфере, **Тобсл** — время обслуживания, **Тпреб** — время в системе, **Ротк** — вероятность отказа, **Кисп** — коэффициент использования прибора;
- на диаграммах места нумеруются с 1, в коде — с 0; «следующее за k» в коде — это (k + 1) mod N.

3. Общий сиквенс

Это главная диаграмма: на ней весь алгоритм модели с развилками. Конкретных чисел здесь нет, только правила. Остальные сиквенсы — частные случаи этой схемы на конкретных данных.

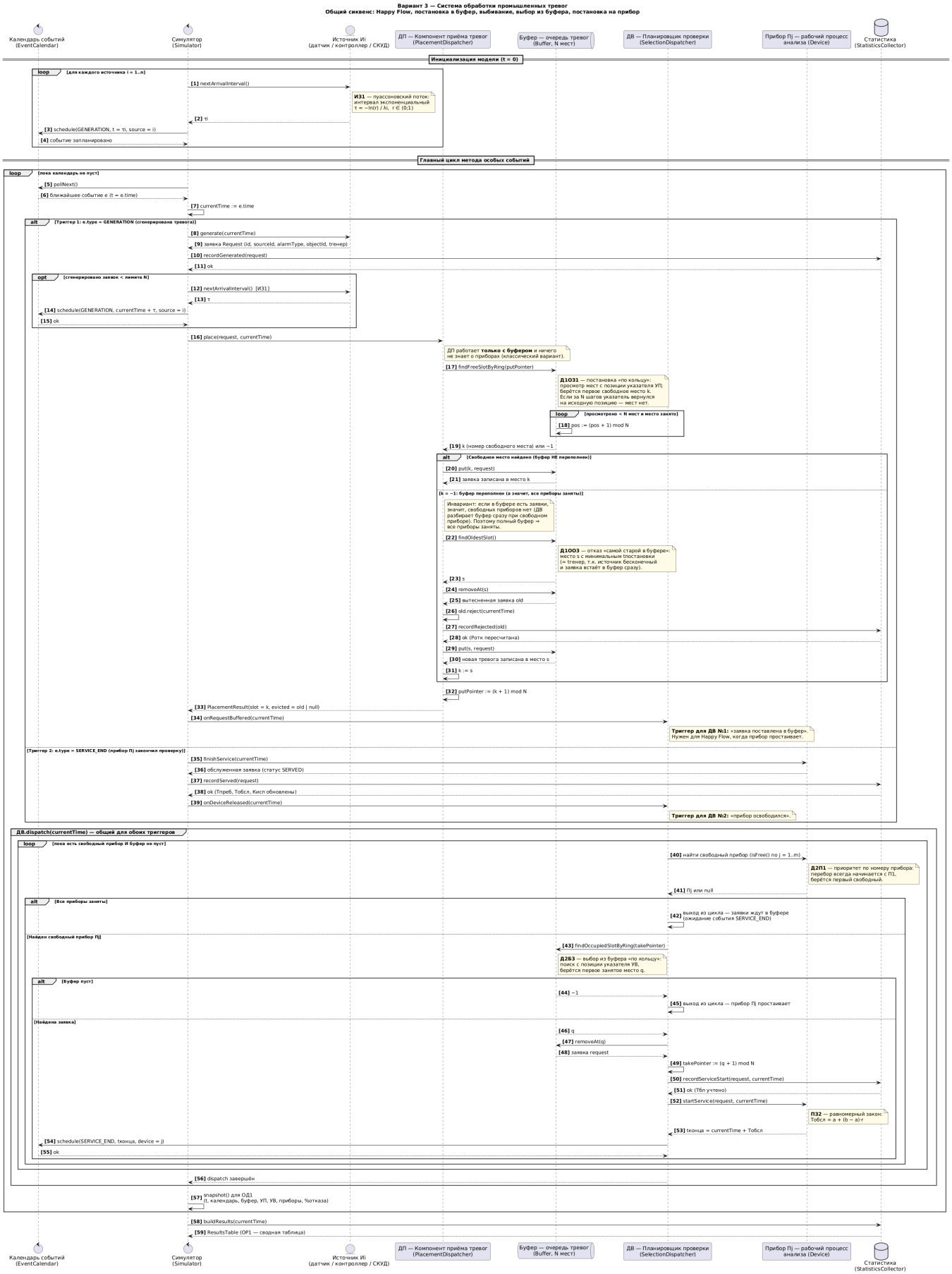


Рис. 1. Общий сиквенс: Happy Flow, постановка, выбивание, выбор из буфера, постановка на прибор

Что происходит по шагам

Шаги	Фаза	Что происходит
[1]–[4]	Инициализация	Для каждого источника симулятор запрашивает первый интервал по ИЗ1 ($\tau = -\ln(r)/\lambda$) и кладёт в календарь событие GENERATION на время τ. После этого в календаре n событий — по одному на источник.
[5]–[7]	Главный цикл	Симулятор берёт из календаря ближайшее событие и переводит модельное время currentTime на время этого события. Цикл повторяется, пока календарь не пуст.
[8]–[11]	Триггер 1: GENERATION	Источник создаёт тревогу (id, источник, тип, объект, время генерации). Статистика увеличивает счётчик сгенерированных заявок.
[12]–[15]	opt: следующая генерация	Если лимит заявок ещё не достигнут, источник считает следующий интервал по ИЗ1 и симулятор планирует следующее GENERATION. Так работает бесконечный источник (ИБ).
[16]–[19]	ДП: поиск места (Д1ОЗ1)	Симулятор передаёт заявку ДП. ДП просит буфер найти свободное место, начиная с УП. Буфер в цикле идёт по кольцу. Если за N шагов вернулся на исходную позицию — мест нет, ответ −1.
[20]–[21]	alt: место есть	ДП кладёт заявку в найденное место k.
[22]–[31]	alt: буфер полон (Д1ОО3)	По инварианту все приборы заняты. ДП находит место s с самой старой заявкой, удаляет её, помечает как REJECTED и сообщает статистике об отказе. Новая заявка встаёт в место s.
[32]–[33]	Сдвиг УП	УП переходит на место, следующее за занятым. ДП возвращает результат: куда поставил и кого выбил (или никого).
[34]	Триггер ДВ №1	Симулятор вызывает ДВ: «заявка поставлена в буфер». Это нужно, чтобы свободный прибор не простаивал при непустом буфере.
[35]–[39]	Триггер 2: SERVICE_END	Прибор завершает проверку и возвращает заявку. Статистика учитывает Тобсл и Тпреб. Симулятор вызывает ДВ: «прибор освободился» (триггер ДВ №2).
[40]–[42]	ДВ: выбор прибора (Д2П1)	Общий для обоих триггеров цикл dispatch. ДВ перебирает приборы с П1. Если все заняты — выход из цикла, заявки ждут в буфере до следующего SERVICE_END.
[43]–[49]	ДВ: выбор заявки (Д2Б3)	Прибор найден, и ДВ ищет занятое место по кольцу с УВ. Если буфер пуст — прибор простаивает. Иначе заявка извлекается из места q, УВ сдвигается на следующее за q.
[50]–[55]	Постановка на прибор (П32)	Статистика фиксирует время в буфере. Прибор начинает проверку, время обслуживания считается по П32. ДВ планирует в календаре SERVICE_END на время окончания.
[56]–[57]	Конец шага	dispatch завершён. Симулятор делает снимок состояния для пошагового режима ОД1.
[58]–[59]	Конец моделирования	Календарь пуст. Статистика строит сводную таблицу ОР1.

Почему цикл dispatch, а не одна выборка? Одно событие может освободить возможность для нескольких назначений. Например, при старте модели несколько приборов свободны одновременно. ДВ повторяет «найти прибор → взять заявку», пока есть и свободный прибор, и заявка.

4. Переполнение буфера → обработка одной заявки

Это обязательный по заданию сценарий: «отказ заявки через переполнение буфера и занятость приборов с последующей постановкой одной заявки на обработку». Здесь он разобран на конкретных числах: буфер на 3 места, 2 прибора, ПЗ2 на [1; 3].

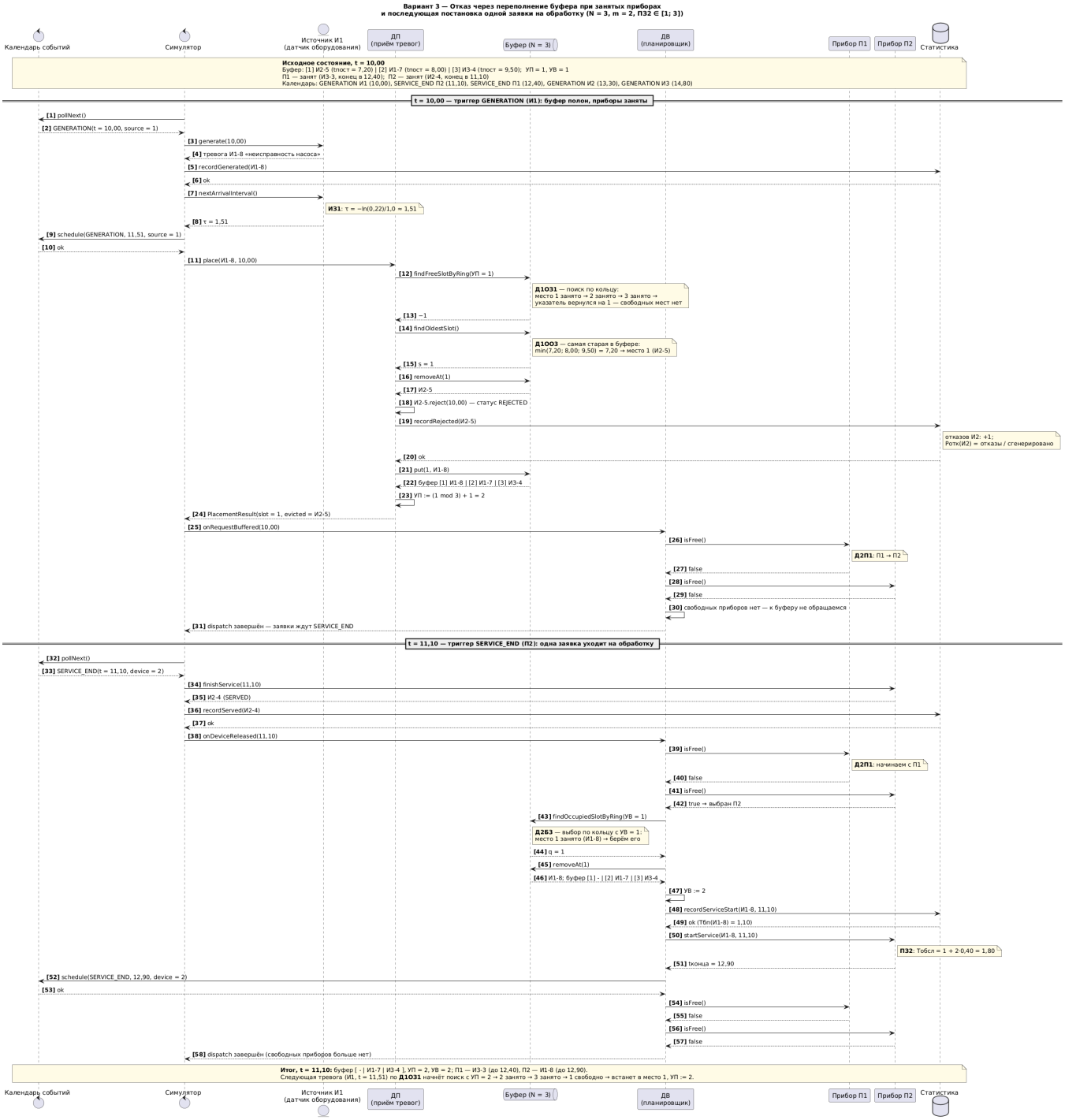


Рис. 2. Отказ через переполнение и последующая постановка заявки на прибор

Исходное состояние, t = 10,00

Место буфера	Заявка	Время постановки
1	И2-5	7,20 (самая старая)
2	И1-7	8,00
3	И3-4	9,50

УП = 1, УВ = 1. П1 обслуживает И3-3 до 12,40, П2 обслуживает И2-4 до 11,10. Ближайшее событие в календаре — GENERATION от И1 в момент 10,00.

Что происходит по шагам

Шаги	Что происходит
[1]–[10]	Календарь выдаёт GENERATION(10,00, И1). Источник И1 создаёт тревогу И1-8 «неисправность насоса», статистика её учитывает. По И31 считается следующий интервал: −ln(0,22)/1,0 ≈ 1,51, следующее GENERATION от И1 планируется на 11,51.
[11]–[13]	ДП ищет свободное место по кольцу (Д1О31) с УП = 1: место 1 занято → 2 занято → 3 занято → указатель вернулся на 1. Свободных мест нет, ответ −1.
[14]–[15]	Срабатывает Д1ОО3: среди 7,20; 8,00; 9,50 минимальное время 7,20, значит самая старая — И2-5 в месте 1.
[16]–[20]	И2-5 удаляется из буфера, получает статус REJECTED, статистика засчитывает отказ источнику И2. Это и есть **отказ**: тревога ушла из системы без проверки.
[21]–[24]	Новая тревога И1-8 встаёт на освободившееся место 1. УП сдвигается на 2. Буфер: [И1-8 | И1-7 | И3-4].
[25]–[31]	Триггер ДВ «заявка поставлена в буфер». ДВ проверяет приборы по Д2П1: П1 занят, П2 занят. К буферу он даже не обращается. Заявки ждут события SERVICE_END.
[32]–[37]	t = 11,10: календарь выдаёт SERVICE_END для П2. П2 отдаёт обслуженную И2-4, статистика её учитывает.
[38]–[42]	Триггер ДВ «прибор освободился». Перебор по Д2П1: П1 занят, П2 свободен — выбран П2.
[43]–[47]	Выбор по кольцу (Д2Б3) с УВ = 1: место 1 занято тревогой И1-8, её и берём. УВ := 2. Обратите внимание: ушла на обработку самая *новая* тревога — это свойство кольцевой дисциплины, а не ошибка.
[48]–[53]	Статистика фиксирует время в буфере Тбп = 11,10 − 10,00 = 1,10. П2 начинает проверку, по П32 Тобсл = 1 + 2·0,40 = 1,80. В календарь ставится SERVICE_END(П2) на 12,90.
[54]–[58]	ДВ снова ищет свободный прибор: оба заняты, цикл завершается.

Итог в момент 11,10: буфер [– \| И1-7 \| И3-4], УП = 2, УВ = 2; П1 обслуживает И3-3 до 12,40, П2 обслуживает И1-8 до 12,90. Следующая тревога (И1, 11,51) начнёт поиск с УП = 2: 2 занято → 3 занято → 1 свободно, встанет в место 1.

5. Happy Flow: буфер свободен, прибор свободен

Самый простой путь тревоги: она приходит, когда очередь пуста и есть свободный прибор. Здесь видно, что даже в этом случае заявка проходит через буфер, но не ждёт в нём (Тбп = 0). Так работает классический ДП, который всегда кладёт заявку в буфер и ничего не знает о приборах.

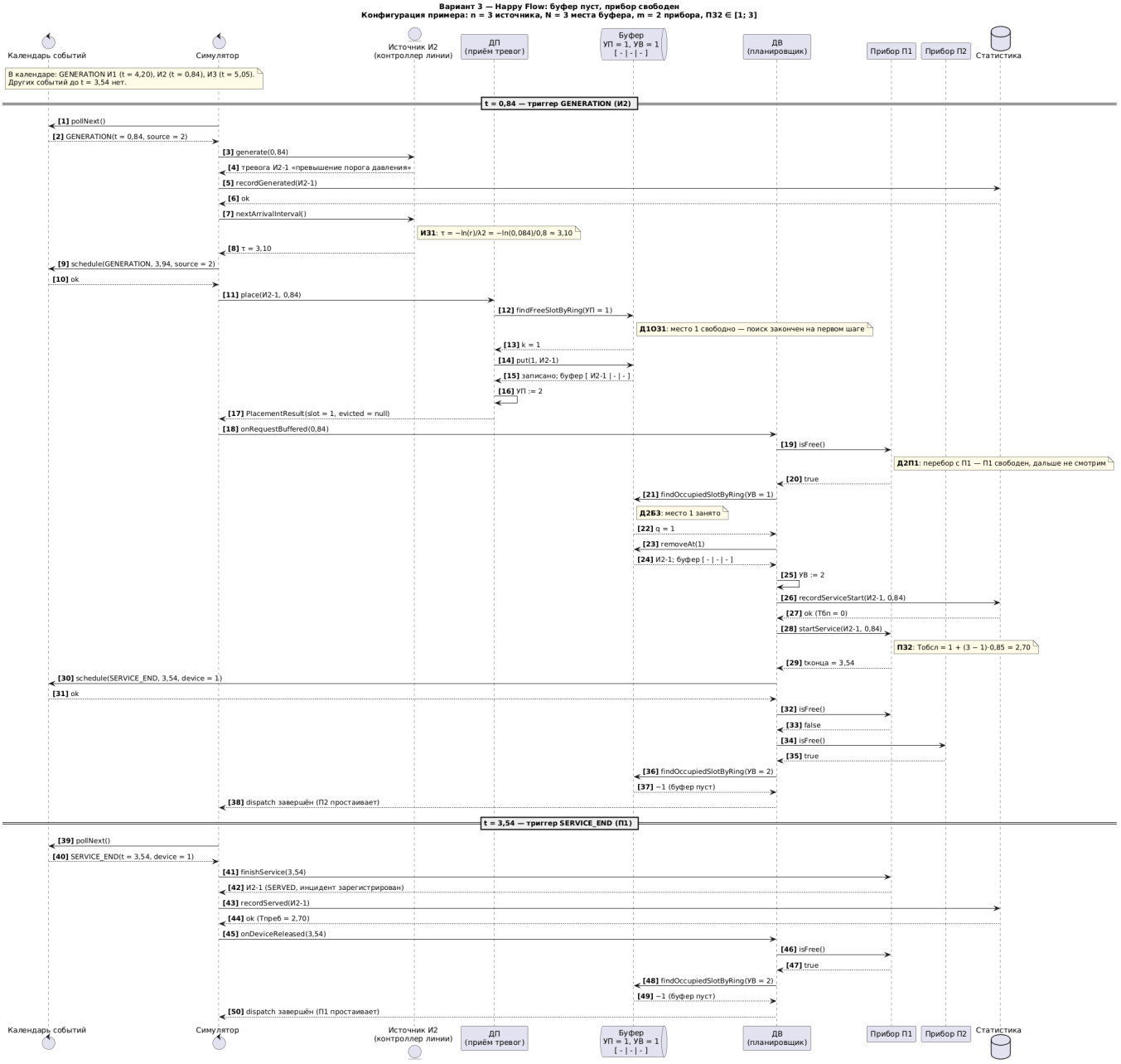


Рис. 3. Happy Flow (n = 3 источника, N = 3 места, m = 2 прибора, ПЗ2 на [1; 3])

Шаги	Что происходит
[1]–[10]	t = 0,84: GENERATION от И2. Создана тревога И2-1 «превышение порога давления». Следующая генерация И2 по И31: −ln(0,084)/0,8 ≈ 3,10, то есть в момент 3,94.
[11]–[17]	ДП ищет место с УП = 1 (Д1О31): место 1 свободно с первого шага. Заявка записана, УП := 2.
[18]–[20]	Триггер ДВ «заявка поставлена в буфер». Д2П1: П1 свободен, дальше приборы не просматриваются.
[21]–[25]	Д2Б3 с УВ = 1: место 1 занято, заявка И2-1 извлекается, УВ := 2. Буфер снова пуст.

Шаги	Что происходит
[26]–[31]	Тбп = 0. П1 начинает проверку, по П32 Тобсл = 1 + 2·0,85 = 2,70. SERVICE_END(П1) запланирован на 3,54.
[32]–[38]	Следующий проход цикла dispatch: П1 занят, П2 свободен, но буфер пуст (−1). П2 простаивает.
[39]–[44]	t = 3,54: SERVICE_END для П1. Тревога И2-1 обработана, инцидент зарегистрирован. Тпреб = 0 + 2,70 = 2,70.
[45]–[50]	Триггер ДВ «прибор освободился»: П1 свободен, но буфер пуст. Оба прибора простаивают до 3,94.

6. Краевые кейсы

По заданию нужно отдельно показать отказ/выбивание, постановку в буфер и выборку из него, постановку на прибор. Здесь все четыре случая собраны на одной диаграмме: буфер на 4 места, 3 прибора.

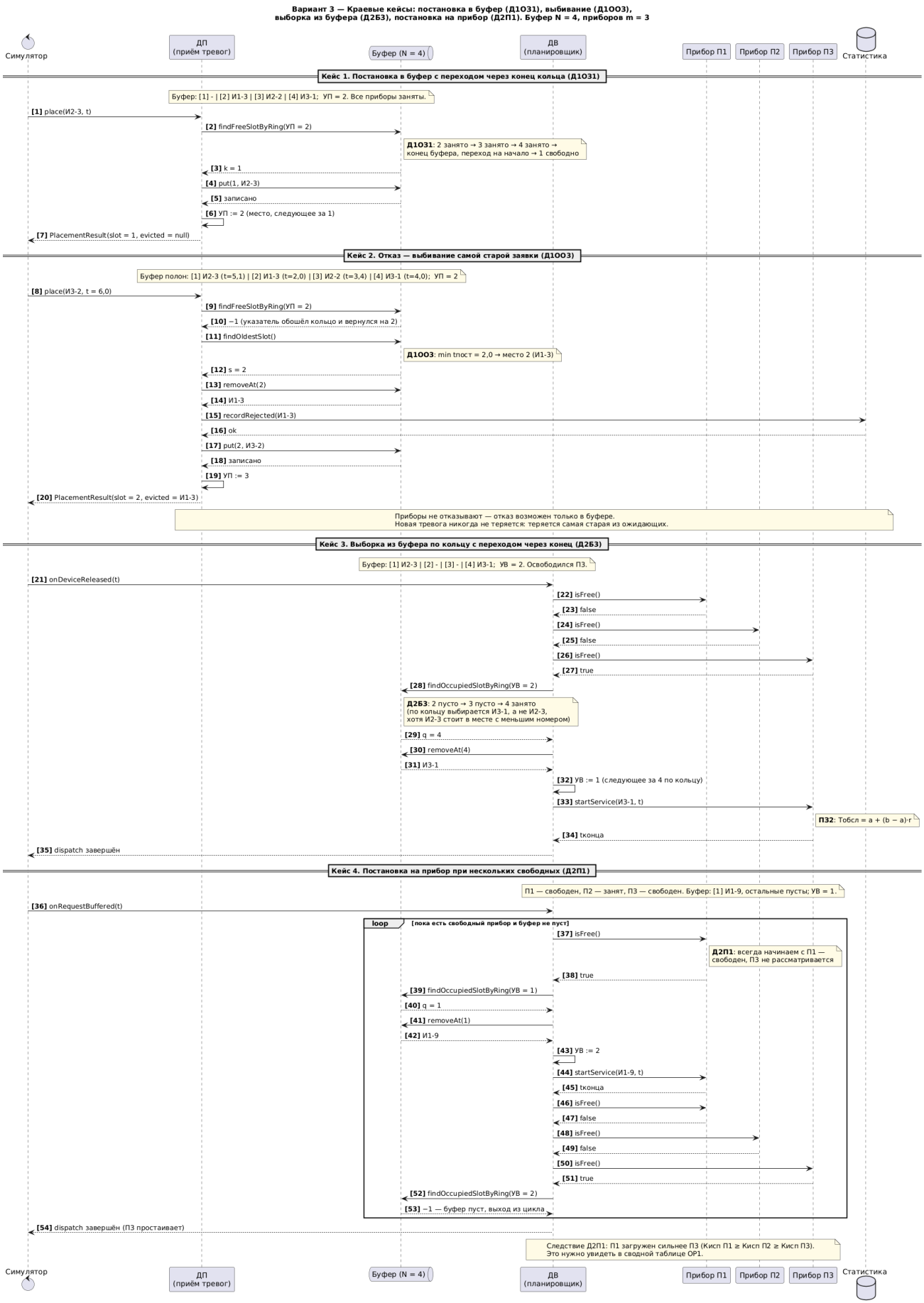


Рис. 4. Краевые кейсы дисциплин варианта

Кейс 1. Постановка с переходом через конец кольца (Д1О31), шаги [1]–[7]

Буфер: [– | И1-3 | И2-2 | И3-1], УП = 2. Поиск идёт 2 → 3 → 4, доходит до конца буфера, переходит на начало и находит свободное место 1. Тревога И2-3 ставится туда, УП := 2. Смысл кейса: указатель не останавливается на конце буфера, а ходит по кругу.

Кейс 2. Выбивание самой старой заявки (Д1ОО3), шаги [8]–[20]

Буфер полон, указатель обошёл кольцо и вернулся на 2 (ответ −1). Среди заявок самая старая — И1-3 (время постановки 2,0), она в месте 2. Она удаляется и засчитывается в отказ, на её место встаёт новая И3-2, УП := 3. Обратите внимание: выбивается не заявка под указателем (это была бы дисциплина Д1ОО1), а именно самая старая по времени.

> Приборы не отказывают, они бывают только заняты. Отказ возможен исключительно в буфере. Новая тревога в нашем варианте не теряется никогда: теряется самая старая из ожидающих.

Кейс 3. Выборка по кольцу с переходом через конец (Д2Б3), шаги [21]–[35]

Буфер: [И2-3 | – | – | И3-1], УВ = 2. Освободился П3, П1 и П2 заняты. Поиск занятого места: 2 пусто → 3 пусто → 4 занято. На обработку уходит И3-1, хотя И2-3 стоит в месте с меньшим номером: поиск начинается с УВ, а не с начала буфера. После выборки УВ := 1 (следующее за 4 по кругу).

Кейс 4. Постановка на прибор при нескольких свободных (Д2П1), шаги [36]–[54]

Свободны П1 и П3, в буфере одна заявка И1-9. Перебор всегда начинается с П1, поэтому заявка уходит на П1, а П3 даже не рассматривается. На втором проходе цикла свободен П3, но буфер пуст — цикл завершается. Следствие дисциплины: приборы с меньшими номерами загружены сильнее (Кисп П1 ≥ Кисп П2 ≥ Кисп П3), и это должно быть видно в сводной таблице ОР1.

7. Диаграмма классов

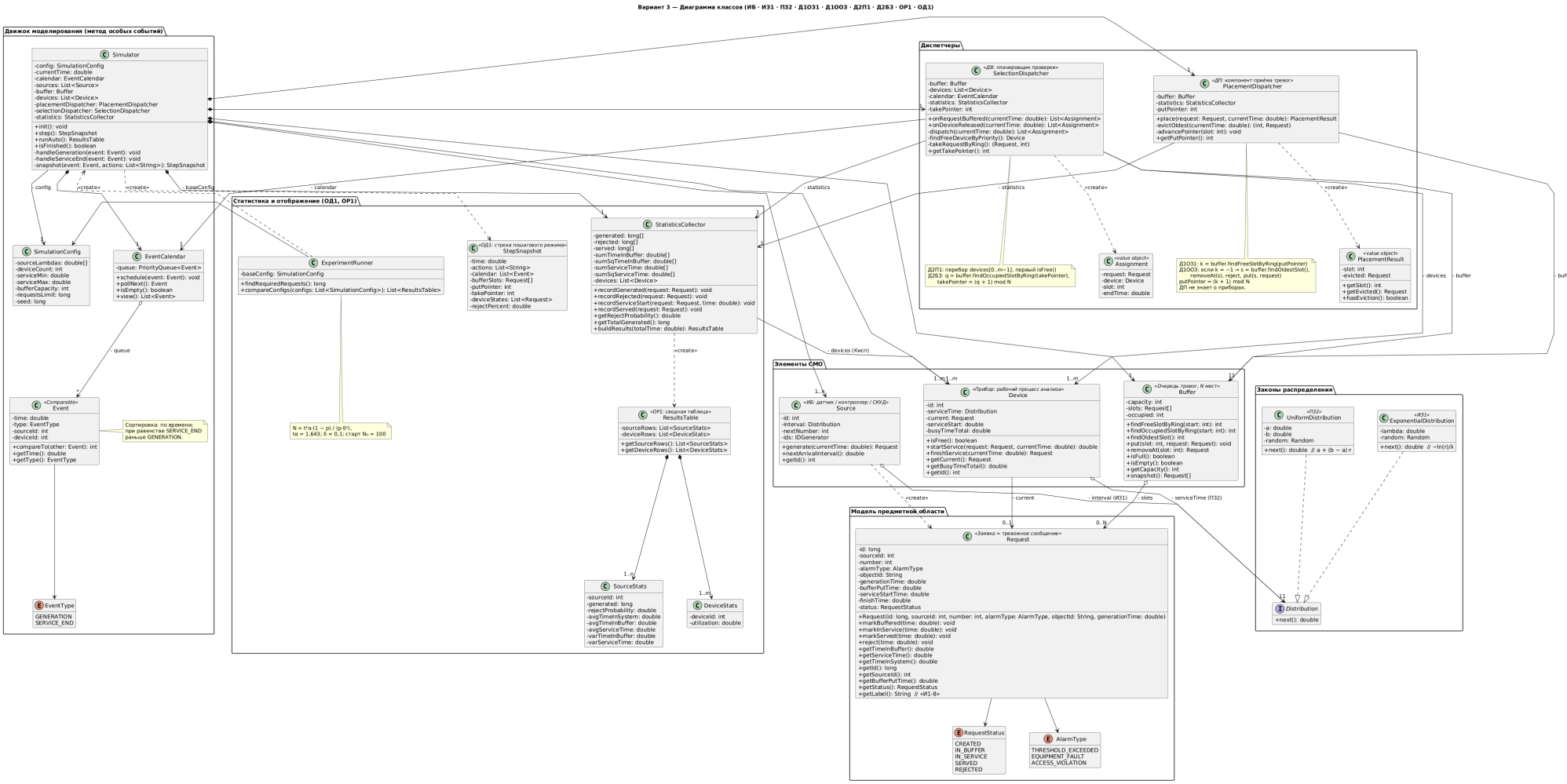


Рис. 5. Диаграмма классов — скелет будущей программы

Что изображено на диаграмме классов

Диаграмма разбита на шесть пакетов. В Go каждый пакет удобно сделать отдельной папкой, каждый класс — структурой с методами.

Пакет / класс	Роль	Ключевые поля и методы
Модель предметной области Request, RequestStatus, AlarmType	Заявка = тревожное сообщение и её жизненный цикл: CREATED → IN_BUFFER → IN_SERVICE → SERVED или REJECTED	Времена генерации, постановки, начала и конца обслуживания; markBuffered, markInService, markServed, reject; геттеры времени в буфере, обслуживания и в системе
Законы распределения Distribution, ExponentialDistribution, UniformDistribution	Генераторы случайных интервалов. Интерфейс позволяет подменять закон и задавать seed для тестов	И31: −ln(r)/λ; П32: a + (b − a)·r
Source	Источник ИБ (датчик, контроллер, СКУД)	generate(t) создаёт Request; nextArrivalInterval() вызывает свой Distribution (И31)
Buffer	Очередь тревог на N мест. Хранит только места, логики дисциплин не принимает	findFreeSlotByRing(start), findOccupiedSlotByRing(start), findOldestSlot(), put, removeAt, snapshot
Device	Прибор — рабочий процесс анализа	startService(req, t) возвращает время окончания по П32; finishService(t) возвращает заявку; копит busyTimeTotal для Кисп
PlacementDispatcher (ДП)	Приём тревог. Работает только с буфером	Приватные: buffer, statistics, putPointer (УП). place() реализует Д1О31 и Д1ОО3
SelectionDispatcher (ДВ)	Планировщик проверки	Приватные: buffer, devices, calendar, statistics, takePointer (УВ). onRequestBuffered и onDeviceReleased — два триггера; dispatch() реализует Д2П1 и Д2Б3
Event, EventCalendar	Календарь особых событий	PriorityQueue по времени; при равном времени SERVICE_END раньше GENERATION
Simulator	Движок метода особых событий, владеет всеми объектами (композиция)	init(), step() → StepSnapshot (ОД1), runAuto() → ResultsTable (ОР1)
StatisticsCollector	Сбор статистики по источникам и приборам	recordGenerated, recordRejected, recordServiceStart, recordServed, buildResults
StepSnapshot, ResultsTable, SourceStats, DeviceStats	Данные для отображения ОД1 и ОР1	Снимок шага: время, календарь, буфер, УП, УВ, приборы, % отказа. Таблица: Ротк, Тпреб, Тбп, Тобсл, Дбп, Добсл, Кисп
ExperimentRunner	Подбор числа заявок и сравнение конфигураций	$N = t^2 \cdot (1 - p)/(p \cdot \delta^2)$, $t\alpha = 1{,}643$, $\delta = 0{,}1$

Почему связи устроены именно так

- **Ссылки — приватные поля.** ДП кладёт заявки в буфер, поэтому у него есть поле «- buffer». ДВ ставит заявки на приборы и планирует их окончание, поэтому у него есть «- devices» и «- calendar». Это прямое требование преподавателя.

- **Методы принимают заявку.** place(request), put(slot, request), startService(request, t) — компоненты общаются через ссылку на объект Request.
- **У ДП нет ссылки на приборы.** Это и есть «классический» ДП, и именно это обеспечивает инвариант «буфер полон ⇒ все приборы заняты».
- **Указатели — приватные поля диспетчеров** (putPointer у ДП, takePointer у ДВ). Буфер только хранит места и даёт примитивы поиска с заданной позиции. Логику прохождения заявки определяют диспетчеры, как в методичке.
- **Simulator владеет всем (◆).** Он создаёт объекты при init() и связывает их между собой. Сам он не решает, куда класть заявку, а только передаёт события нужному диспетчеру.
- **Инкапсуляция.** Внутренние вспомогательные методы приватные: evictOldest, advancePointer, dispatch, findFreeDeviceByPriority, takeRequestByRing.

8. Flowchart с дорожками

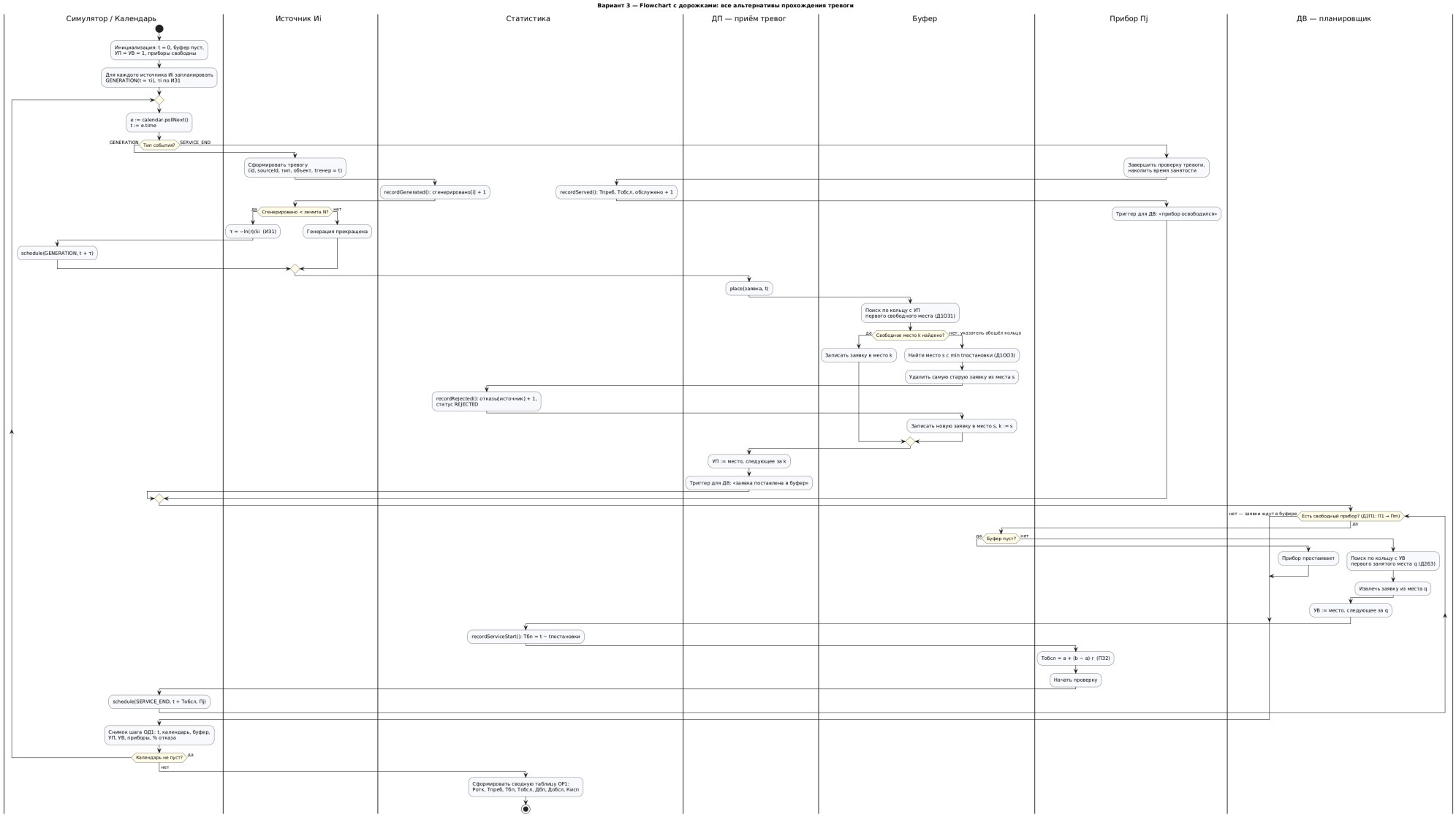


Рис. 6. Блок-схема алгоритма с дорожками по участникам

Как читать flowchart

Это тот же алгоритм, что на общем сиквенсе, но в виде блок-схемы. Каждая вертикальная колонка (дорожка) — участник, который выполняет действие. Ромбы — развилки, по ним видны все альтернативы. Когда стрелка переходит из одной дорожки в другую, работа передаётся другому участнику.

Дорожка	Что в ней происходит
Симулятор / Календарь	Инициализация, главный цикл «взять ближайшее событие», развилка по типу события, планирование событий, снимок шага ОД1, проверка «календарь не пуст?»
Источник Иi	Формирование тревоги; развилка «лимит заявок не достигнут?» → расчёт следующего интервала по ИЗ1 или прекращение генерации
ДП — приём тревог	Вызов place(); сдвиг УП; триггер для ДВ «заявка поставлена в буфер»
Буфер	Поиск свободного места по кольцу (Д1О31); развилка «место найдено?»: запись, либо поиск самой старой (Д1ОО3) и её удаление; развилка «буфер пуст?» при выборке; поиск по кольцу (Д2Б3)
Прибор Пj	Завершение проверки и триггер «прибор освободился»; расчёт Тобсл по П32 и начало проверки
ДВ — планировщик	Цикл «есть свободный прибор? (Д2П1)»; сдвиг УВ; выход из цикла, когда приборы заняты или буфер пуст
Статистика	Учёт сгенерированных, отказов, времени в буфере, обслуженных; в конце — сводная таблица ОР1

Все альтернативы, которые отражены на схеме

- тип события: GENERATION или SERVICE_END;
- лимит заявок: продолжить генерацию или остановить источник;
- буфер: есть свободное место или переполнен (выбивание самой старой);
- приборы: есть свободный или все заняты (заявки ждут);
- выборка: буфер пуст (прибор простаивает) или есть заявка (постановка на прибор);
- завершение: календарь не пуст (следующий шаг) или пуст (сводная таблица и конец).

9. Бизнес-процесс в ArchiMate 3.0

Необязательная по заданию диаграмма. Она показывает систему не глазами программиста, а глазами бизнеса: какие события и процессы происходят на заводе и какие программные компоненты их выполняют.

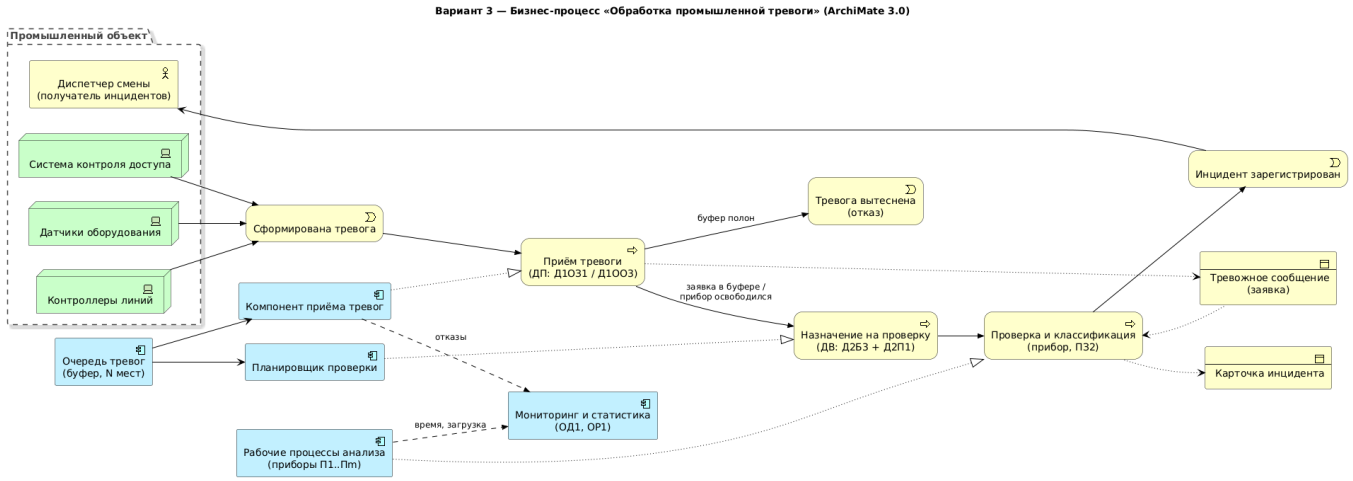


Рис. 7. Бизнес-процесс «Обработка промышленной тревоги»

Элемент	Цвет / значок	Что означает
Датчики, контроллеры, СКУД	Зелёные «устройства»	Технологический уровень: источники тревог
Сформирована тревога	Жёлтое событие	Бизнес-событие, которое запускает процесс (= GENERATION)
Приём → Назначение → Проверка	Жёлтые процессы со стрелкой	Цепочка бизнес-процессов, соответствует ДП → ДВ → прибор
Тревога вытеснена (отказ)	Жёлтое событие	Альтернативный исход приёма при полном буфере (Д1ОО3)
Инцидент зарегистрирован	Жёлтое событие	Нормальный исход, обслуживает диспетчера смены
Тревожное сообщение, карточка инцидента	Бизнес-объекты	Данные, которые процессы читают и пишут
Компонент приёма, планировщик, рабочие процессы, очередь, мониторинг	Голубые компоненты	Прикладной уровень: программные компоненты, которые *реализуют* процессы (пунктир с треугольником)

Сплошная стрелка — triggering (одно запускает другое), пунктир с треугольником — realization (компонент реализует процесс), пунктирная стрелка — flow (передача данных: отказы, время, загрузка).

10. Вопросы на защите и короткие ответы

Вопрос	Ответ
Что запускает диспетчер постановки?	Событие GENERATION: источник сгенерировал тревогу. Сам по себе ДП не работает.
Что запускает диспетчер выбора?	Два события: «заявка поставлена в буфер» (иначе в Happy Flow свободный прибор не получит заявку) и «прибор освободился» (SERVICE_END).
Почему ДП не проверяет приборы перед отказом?	Инвариант: ДВ разбирает буфер сразу при свободном приборе, поэтому если буфер полон, все приборы уже заняты.
Где может произойти отказ?	Только в буфере. Приборы не отказывают, они бывают заняты. В варианте 3 отказ получает самая старая заявка в буфере (Д1ООЗ), а новая встаёт на её место.
Чем Д1ОЗ1 отличается от Д1ОЗЗ?	Д1О31 ищет свободное место с указателя и идёт по кругу. Д1ОЗЗ всегда ищет с начала буфера, указателя нет.
Почему на обработку может уйти самая новая заявка?	Д2БЗ выбирает первую занятую позицию с указателя УВ, а не самую старую. Если новая заявка встала как раз под УВ, её возьмут первой. Это свойство дисциплины.
Как выбирается прибор?	Д2П1: перебор всегда с П1, берётся первый свободный. Поэтому П1 загружен сильнее остальных.
Что будет при одновременных событиях?	Сначала SERVICE_END, потом GENERATION. Освободившийся прибор успеет забрать заявку из буфера до прихода новой, лишнего отказа не будет.
Как определяется «самая старая»?	По времени постановки в буфер. При бесконечном источнике заявка попадает в буфер в момент генерации, поэтому это совпадает со временем генерации.
Где хранятся указатели и зачем они в ОД1?	УП — приватное поле ДП, УВ — приватное поле ДВ. В пошаговом режиме их положение нужно показывать на каждом шаге, поэтому у диспетчеров есть геттеры.
Когда собирается статистика?	Генерация → recordGenerated; выбивание → recordRejected; постановка на прибор → время в буфере; окончание обслуживания → время обслуживания и в системе; конец моделирования → Кисп и сводная таблица.
Сколько заявок нужно прогнать?	$N = t^2 \cdot (1 - p)/(p \cdot \delta^2)$ при $t\alpha = 1{,}643$ и $\delta = 0{,}1$. Сначала пристрелка $N_0 = 100$, по ней оценивается p, затем N пересчитывается, пока p не стабилизируется.
Зонный или общий буфер?	Зонный — когда источники неравноправны и один может «задавить» других (например, тревоги СКУД не должны вытесняться телеметрией). Общий — когда источники однородны и нужна максимальная утилизация мест. В варианте 3 буфер общий.
Как диаграммы связаны с кодом?	Каждый класс диаграммы станет структурой в Go, каждый вызов на сиквенсе — вызовом метода. Сценарии с сиквенсов удобно оформить как тесты: подать то же исходное состояние и проверить тот же результат.